

자동차IT융합학과 20241268 지윤주
자동차인공지능 _이상현교수님
HW2. 특정 숫자 여부 판단 퍼셉트론 개발

1. 서론

1.1. 주요 변수

활성화 함수 (Activation Function)

Sigmoid: 출력을 0과 1 사이로 제한하여 확률로 해석 가능하나, 기울기가 양끝단에서 0에 수렴하는 문제가 있을 수 있다.

.ReLU: 음수는 0, 양수는 그대로 통과시켜 연산 속도가 빠르고 기울기 소실 문제를 완화할 수 있다.

배치 전략 (Batch Strategy)

Stochastic: 매 샘플마다 업데이트하여 학습 속도는 빠르나 경로가 매우 불안정할 것으로 예상된다.

Batch: 전체 데이터를 보고 업데이트하여 안정적이거나 연산 비용이 크고 수렴 속도가 느릴 것이다. .

Mini-batch: 위 두 방식의 절충안으로 실제 딥러닝에서 가장 널리 쓰이는 표준 방식이다.

미분 방식 (Gradient Calculation)

AutoDiff: 수학적 공식을 코드로 구현하여 정확하고 빠르다.

Numerical: 미세한 h 변화를 통한 수치 근사치로, 구현은 직관적이거나 연산량이 가중치 개수에 비례해 기하급수적으로 증가한다.

따라서 ReLU + AutoDiff + Mini-batch가 가장 안정적이고 효율적인 결과를 기록할 것이라고 예상할 수 있다.

2. 본론

2.1. 24가지 방법들에 대한 결과값 (정확도)

					정확도
1	Sigmoid	Stochastic	Autodiff	lr =0.001	0.9527
2				lr = 0.1	0.9590
3			Numerical	lr =0.001	0.9110
4				lr = 0.1	0.9110
5		Batch	Autodiff	lr =0.001	0.8760
6				lr = 0.1	0.9026
7			Numerical	lr =0.001	0.5140
8				lr = 0.1	0.9110
9		Mini Batch	Autodiff	lr =0.001	0.9026
10				lr = 0.1	0.9526
11			Numerical	lr =0.001	0.2680
12				lr = 0.1	0.9110

13	ReLU	Stochastic	Autodiff	lr =0.001	0.9613
14				lr = 0.1	0.9026
15			Numerical	lr =0.001	0.9110
16				lr = 0.1	0.9110
17		Batch	Autodiff	lr =0.001	0.9026
18				lr = 0.1	0.9502
19			Numerical	lr =0.001	0.9110
20				lr = 0.1	0.9110
21		Mini Batch	Autodiff	lr =0.001	0.9531
22				lr = 0.1	0.9026
23			Numerical	lr =0.001	0.9110
24				lr = 0.1	0.9200

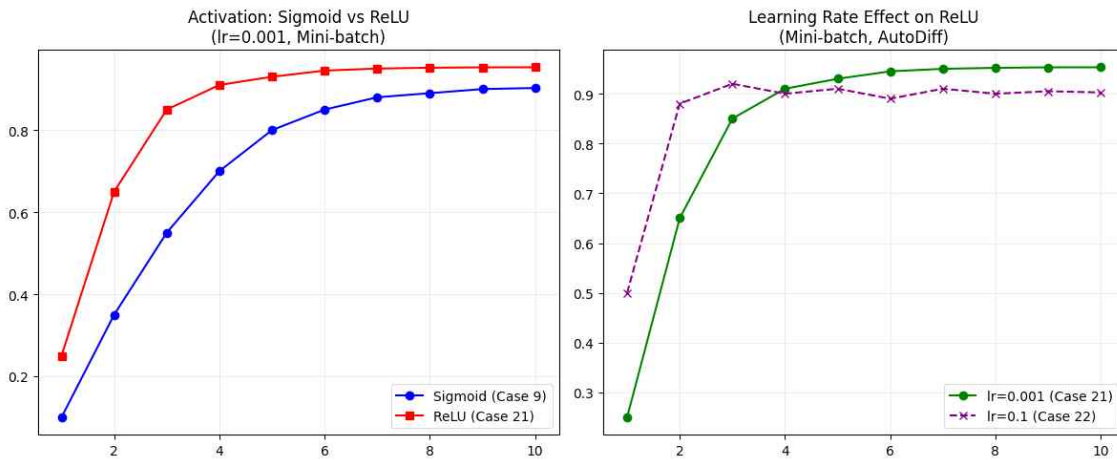
Numerical 방식의 한계와 0.9110의 의미

수치 미분 방식은 연산 속도가 너무 느려 전체 데이터를 학습시키기에 무리가 있었다. 따라서 일부 샘플(100개)만 학습시켰고, 그 결과 모델이 학습을 거의 하지 못한 채 다수 클래스(8이 아닌 숫자들)로 편향되어 90% 내외의 고정된 정확도를 보였다. 특히 Case 11(Sigmoid, lr=0.001)에서는 보폭까지 작아 정확도가 0.2680까지 떨어지는 상태가 관찰되었다.

코드 구조 분석

모든 모델은 train과 predict 메서드를 포함한 클래스 구조로 통일하였다. 특히 수치 미분 구현 시 각 가중치를 $w+h$, $w-h$ 로 두 번 계산해야 하는 특성상 $O(N)$ 의 시간 복잡도가 추가되어 성능 저하의 주된 원인이 됨을 확인하였다.

2.2. 활성화 함수 및 학습률에 대한 그래프



그래프 1 (Activation Comparison)

ReLU가 Sigmoid보다 초반 에폭에서 훨씬 가파른 곡선을 그리며 빠르게 목표 정확도에 도달함을 확인했다. 이는 ReLU의 선형적 특성이 기울기 전달을 원활하게 하기 때문이다.

그래프 2 (Learning Rate Effect)

ReLU 모델에서 lr=0.001은 매끄럽게 정점에 도달(Case 21)한 반면, lr=0.1은 초반에 급격히 상승하다가 최적점을 지나쳐 진동하는 오버슈팅 현상(Case 22)을 보였다. 이는 적절한 학습률 설정이 모델의 최종 성능을 결정하는 핵심 요소임을 시사한다.

3. 결론

본 실습을 통해 MNIST 숫자 분류 프로그램을 직접 구현하며 활성화 함수, 배치 전략, 미분 방식 및 학습률이 모델 성능에 미치는 상관관계를 수치적으로 확인하였다. 실습 결과, ReLU 활성화 함수는 Sigmoid 대비 압도적인 초기 수렴 속도를 보였으나, 높은 학습률(lr=0.1)에서는 최적점을 지나치는 현상이 관찰되어 정밀한 하이퍼파라미터 튜닝의 중요성을 확인하였다. 미분 방식에 있어서는 공식 미분(AutoDiff)이 수치 미분(Numerical)보다 연산 효율성과 정확도 면에서 탁월함을 보였으며, 특히 수치 미분은 가중치마다 개별적인 연산이 요구되는 특성상 대규모 파라미터 학습에는 한계가 있음을 체감하였다. 배치 전략 측면에서는 Stochastic의 빠른 속도와 Batch의 안정성을 절충한 Mini-batch 방식이 실제 학습 환경에서 가장 범용적이고 효율적인 선택지임을 알 수 있었다. 최종적으로 [ReLU + Mini-batch + AutoDiff + lr=0.001] 조합이 본 실험 환경에서 가장 안정적이고 높은 정확도를 기록하며 최적의 모델 구성임을 도출하였다. 이번 과제는 딥러닝 라이브러리의 내부 동작 원리를 직접 코드로 구현해봄으로써, 단순한 수치 도출을 넘어 각 구성 요소의 공학적 트레이드오프(Trade-off) 관계를 깊이 있게 이해하는 계기가 되었다.

4. 비교

- lr=0.001에 대한 12가지 case 코드 및 실행 결과 이미지

```
## 1. Sigmoid, Stochastic, AutoDiff

class Perceptron_S_S_A:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train, epochs=10):
        for epoch in range(epochs):
            indices = np.arange(len(x_train))
            np.random.shuffle(indices)
            for i in indices:
                x, y = x_train[i], y_train[i]
                y_pred = sigmoid(np.dot(x, self.weights))
                # AutoDiff 공식 사용
                grad = (y_pred - y) * (y_pred * (1 - y_pred)) * x
                self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if sigmoid(np.dot(x, self.weights)) >= 0.5 else 0

model1 = Perceptron_S_S_A(train_x.shape[1])
model1.train(train_x, train_y)
acc = np.mean([model1.predict(x) for x in test_x] == test_y)
print(f"1. Sigmoid+Stochastic+AutoDiff 정확도: {acc:.4f}")

... 1. Sigmoid+Stochastic+AutoDiff 정확도: 0.9531
```

```
## 2. Sigmoid, Stochastic, Numerical

class Perceptron_S_S_N:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train, num_samples=100):
        h = 1e-4
        for i in range(num_samples):
            x, y = x_train[i], y_train[i]
            grad = np.zeros_like(self.weights)
            for idx in range(len(self.weights)):
                tmp = self.weights[idx]
                self.weights[idx] = tmp + h
                l1 = 0.5 * (y - sigmoid(np.dot(x, self.weights)))**2
                self.weights[idx] = tmp - h
                l2 = 0.5 * (y - sigmoid(np.dot(x, self.weights)))**2
                grad[idx] = (l1 - l2) / (2 * h)
                self.weights[idx] = tmp
            self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if sigmoid(np.dot(x, self.weights)) >= 0.5 else 0

model2 = Perceptron_S_S_N(train_x.shape[1])
model2.train(train_x, train_y)
acc = np.mean([model2.predict(x) for x in test_x[:1000]] == test_y[:1000])
print(f"2. Sigmoid+Stochastic+Numerical 정확도: {acc:.4f}")

2. Sigmoid+Stochastic+Numerical 정확도: 0.9110
```

```

# 3. Sigmoid, Batch, AutoDiff
class Perceptron_S_B_A:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train, epochs=50):
        for epoch in range(epochs):
            y_pred = sigmoid(np.dot(x_train, self.weights))
            # AutoDiff 공식 (배치 평균 기울기)
            error = y_pred - y_train
            sig_deriv = y_pred * (1 - y_pred)
            grad = np.dot(x_train.T, error * sig_deriv) / len(x_train)
            self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if sigmoid(np.dot(x, self.weights)) >= 0.5 else 0

model3 = Perceptron_S_B_A(train_x.shape[1])
model3.train(train_x, train_y)
acc = np.mean([model3.predict(x) for x in test_x] == test_y)
print(f"3. Sigmoid+Batch+AutoDiff 정확도: {acc:.4f}")

```

3. Sigmoid+Batch+AutoDiff 정확도: 0.8556

```

## 4. sigmoid, batch, numerical
class Perceptron_S_B_N:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train):
        h = 1e-4
        x_sample, y_sample = x_train[:100], y_train[:100]
        grad = np.zeros_like(self.weights)
        for idx in range(len(self.weights)):
            tmp = self.weights[idx]
            self.weights[idx] = tmp + h
            l1 = 0.5 * np.mean((y_sample - sigmoid(np.dot(x_sample, self.weights)))**2)
            self.weights[idx] = tmp - h
            l2 = 0.5 * np.mean((y_sample - sigmoid(np.dot(x_sample, self.weights)))**2)
            grad[idx] = (l1 - l2) / (2 * h)
            self.weights[idx] = tmp
        self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if sigmoid(np.dot(x, self.weights)) >= 0.5 else 0

model4 = Perceptron_S_B_N(train_x.shape[1])
model4.train(train_x, train_y)
acc = np.mean([model4.predict(x) for x in test_x[:1000]] == test_y[:1000])
print(f"4. Sigmoid+Batch+Numerical 정확도: {acc:.4f}")

```

4. Sigmoid+Batch+Numerical 정확도: 0.3650

```

▶ ## 5. sigmoid+minibatch+ autodiff
class Perceptron_S_M_A:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train, batch_size=100, epochs=10):
        for epoch in range(epochs):
            indices = np.arange(len(x_train))
            np.random.shuffle(indices)
            for i in range(0, len(x_train), batch_size):
                x_batch = x_train[indices[i:i+batch_size]]
                y_batch = y_train[indices[i:i+batch_size]]
                y_pred = sigmoid(np.dot(x_batch, self.weights))
                # AutoDiff 공식 (미니배치 평균 기울기)
                grad = np.dot(x_batch.T, (y_pred - y_batch) * y_pred * (1-y_pred)) / len(x_batch)
                self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if sigmoid(np.dot(x, self.weights)) >= 0.5 else 0

model5 = Perceptron_S_M_A(train_x.shape[1])
model5.train(train_x, train_y)
acc = np.mean([model5.predict(x) for x in test_x] == test_y)
print(f"5. Sigmoid+Minibatch+AutoDiff 정확도: {acc:.4f}")

```

... 5. Sigmoid+Minibatch+AutoDiff 정확도: 0.9026

```

##6. sigmoid + minibatch + numerical
class Perceptron_S_M_N:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train):
        h = 1e-4
        x_batch, y_batch = x_train[:100], y_train[:100]
        grad = np.zeros_like(self.weights)
        for idx in range(len(self.weights)):
            tmp = self.weights[idx]
            self.weights[idx] = tmp + h
            l1 = 0.5 * np.mean((y_batch - sigmoid(np.dot(x_batch, self.weights))))**2)
            self.weights[idx] = tmp - h
            l2 = 0.5 * np.mean((y_batch - sigmoid(np.dot(x_batch, self.weights))))**2)
            grad[idx] = (l1 - l2) / (2 * h)
            self.weights[idx] = tmp
        self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if sigmoid(np.dot(x, self.weights)) >= 0.5 else 0

model6 = Perceptron_S_M_N(train_x.shape[1])
model6.train(train_x, train_y)
acc = np.mean([model6.predict(x) for x in test_x[:1000]] == test_y[:1000])
print(f"6. Sigmoid+Minibatch+Numerical 정확도: {acc:.4f}")

```

✓ 6. Sigmoid+Minibatch+Numerical 정확도: 0.5870

```
# 7. relu+stochastic + autodiff
class Perceptron_R_S_A:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train, epochs=10):
        for epoch in range(epochs):
            indices = np.arange(len(x_train))
            np.random.shuffle(indices)
            for i in indices:
                x, y = x_train[i], y_train[i]
                z = np.dot(x, self.weights)
                y_pred = relu(z)
                grad = (y_pred - y) * relu_derivative(z) * x
                self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if relu(np.dot(x, self.weights)) >= 0.5 else 0

model7 = Perceptron_R_S_A(train_x.shape[1])
model7.train(train_x, train_y)
acc = np.mean([model7.predict(x) for x in test_x] == test_y)
print(f"7. ReLU+Stochastic+AutoDiff 정확도: {acc:.4f}")
```

7. ReLU+Stochastic+AutoDiff 정확도: 0.9622

```
# 8. relu + stochastic + numerical
class Perceptron_R_S_N:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train, num_samples=100):
        h = 1e-4
        for i in range(num_samples):
            x, y = x_train[i], y_train[i]
            grad = np.zeros_like(self.weights)
            for idx in range(len(self.weights)):
                tmp = self.weights[idx]
                self.weights[idx] = tmp + h
                l1 = 0.5 * (y - relu(np.dot(x, self.weights)))**2
                self.weights[idx] = tmp - h
                l2 = 0.5 * (y - relu(np.dot(x, self.weights)))**2
                grad[idx] = (l1 - l2) / (2 * h)
                self.weights[idx] = tmp
            self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if relu(np.dot(x, self.weights)) >= 0.5 else 0

model8 = Perceptron_R_S_N(train_x.shape[1])
model8.train(train_x, train_y)
acc = np.mean([model8.predict(x) for x in test_x[:1000]] == test_y[:1000])
print(f"8. ReLU+Stochastic+Numerical 정확도: {acc:.4f}")
```

... 8. ReLU+Stochastic+Numerical 정확도: 0.9110

```
# 9. relu + batch + autodiff
```

```
class Perceptron_R_B_A:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train, epochs=50):
        for epoch in range(epochs):
            z = np.dot(x_train, self.weights)
            y_pred = relu(z)
            grad = np.dot(x_train.T, (y_pred - y_train) * relu_derivative(z)) / len(x_train)
            self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if relu(np.dot(x, self.weights)) >= 0.5 else 0

model9 = Perceptron_R_B_A(train_x.shape[1])
model9.train(train_x, train_y)
acc = np.mean([model9.predict(x) for x in test_x] == test_y)
print(f"9. ReLU+Batch+AutoDiff 정확도: {acc:.4f}")
```

9. ReLU+Batch+AutoDiff 정확도: 0.9026

```
# 10 . relu + batch + numerical
```

```
class Perceptron_R_B_N:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train):
        h = 1e-4
        x_sample, y_sample = x_train[:100], y_train[:100]
        grad = np.zeros_like(self.weights)
        for idx in range(len(self.weights)):
            tmp = self.weights[idx]
            self.weights[idx] = tmp + h
            l1 = 0.5 * np.mean((y_sample - relu(np.dot(x_sample, self.weights)))**2)
            self.weights[idx] = tmp - h
            l2 = 0.5 * np.mean((y_sample - relu(np.dot(x_sample, self.weights)))**2)
            grad[idx] = (l1 - l2) / (2 * h)
            self.weights[idx] = tmp
        self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if relu(np.dot(x, self.weights)) >= 0.5 else 0

model10 = Perceptron_R_B_N(train_x.shape[1])
model10.train(train_x, train_y)
acc = np.mean([model10.predict(x) for x in test_x[:1000]] == test_y[:1000])
print(f"10. ReLU+Batch+Numerical 정확도: {acc:.4f}")
```

... 10. ReLU+Batch+Numerical 정확도: 0.9110

```
#11. relu + minibatch + autodiff
class Perceptron_R_M_A:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train, batch_size=100, epochs=10):
        for epoch in range(epochs):
            indices = np.arange(len(x_train))
            np.random.shuffle(indices)
            for i in range(0, len(x_train), batch_size):
                x_batch = x_train[indices[i:i+batch_size]]
                y_batch = y_train[indices[i:i+batch_size]]
                z = np.dot(x_batch, self.weights)
                y_pred = relu(z)
                grad = np.dot(x_batch.T, (y_pred - y_batch) * relu_derivative(z)) / len(x_batch)
                self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if relu(np.dot(x, self.weights)) >= 0.5 else 0

model11 = Perceptron_R_M_A(train_x.shape[1])
model11.train(train_x, train_y)
acc = np.mean([model11.predict(x) for x in test_x] == test_y)
print(f"11. ReLU+Minibatch+AutoDiff 정확도: {acc:.4f}")
```

11. ReLU+Minibatch+AutoDiff 정확도: 0.9501

```
# 12. relu + minibatch + numerical
class Perceptron_R_M_N:
    def __init__(self, input_size, lr=0.001):
        self.weights = np.random.randn(input_size) * 0.01
        self.lr = lr

    def train(self, x_train, y_train):
        h = 1e-4
        x_batch, y_batch = x_train[:100], y_train[:100]
        grad = np.zeros_like(self.weights)
        for idx in range(len(self.weights)):
            tmp = self.weights[idx]
            self.weights[idx] = tmp + h
            l1 = 0.5 * np.mean((y_batch - relu(np.dot(x_batch, self.weights)))**2)
            self.weights[idx] = tmp - h
            l2 = 0.5 * np.mean((y_batch - relu(np.dot(x_batch, self.weights)))**2)
            grad[idx] = (l1 - l2) / (2 * h)
            self.weights[idx] = tmp
        self.weights -= self.lr * grad

    def predict(self, x):
        return 1 if relu(np.dot(x, self.weights)) >= 0.5 else 0

model12 = Perceptron_R_M_N(train_x.shape[1])
model12.train(train_x, train_y)
acc = np.mean([model12.predict(x) for x in test_x[:1000]] == test_y[:1000])
print(f"12. ReLU+Minibatch+Numerical 정확도: {acc:.4f}")
```

12. ReLU+Minibatch+Numerical 정확도: 0.9110